

Deadlock Detection Simulator

Operating Systems – Lab Project Report
Implemented in C++ | Console-Based Application

Subject	Operating Systems
Language	C++ (Standard: C++11 or later)
Paradigm	Console / Command-Line Application
Algorithm	Resource-Allocation Graph + Cycle Detection
Platform	Windows / Linux / macOS (any terminal)

1. Introduction

In a multiprogramming operating system, multiple processes compete for a finite set of resources such as memory, CPU time, and I/O devices. A **deadlock** occurs when a group of processes are each waiting for a resource that is held by another process in the group, forming a circular wait from which none can escape without external intervention.

Deadlocks are one of the most studied problems in operating systems because they can silently bring an entire system to a halt. Unlike errors that cause crashes, a deadlocked system continues to run — it simply stops making progress. This makes detection both critically important and non-trivial.

This project implements a **Deadlock Detection Simulator** in C++. The program models a system's state through allocation and request matrices, then applies a graph-reduction / work-vector algorithm to determine whether a deadlock exists and, if so, which processes are involved.

2. Objective

The primary objectives of this project are:

- Understand and implement the deadlock detection algorithm based on the Resource Allocation Graph (RAG) model.
- Allow the user to enter a custom system state (allocation matrix, request matrix, available resources) interactively.
- Provide built-in demo scenarios so the user can observe deadlocked, partially deadlocked, and safe states without manual entry.
- Clearly report which processes are deadlocked and suggest possible resolution strategies.
- Serve as a hands-on learning tool for Operating Systems coursework.

3. Background / Theory

3.1 Deadlock Conditions (Coffman, 1971)

A deadlock can arise if and only if all four of the following conditions hold simultaneously:

Mutual Exclusion	At least one resource must be held in a non-shareable mode.
Hold and Wait	A process holding one resource is waiting to acquire more.
No Preemption	Resources cannot be forcibly taken away from a process.
Circular Wait	A cycle exists in the process-resource wait graph.

3.2 Detection Algorithm (Work-Vector Method)

The simulator uses a **work-vector algorithm** (closely related to Banker's Algorithm safety check) to detect cycles in the resource allocation state:

- Step 1. Initialize **Work** = Available vector.
- Step 2. Set **Finish[i] = False** for all processes i.
- Step 3. Find a process i such that Finish[i] = False and Request[i] <= Work.
- Step 4. If found: Work = Work + Allocation[i], set Finish[i] = True, go to step 3.
- Step 5. If no such process exists: any process with Finish[i] = False is deadlocked.

This algorithm runs in $O(n * m)$ time where n is the number of processes and m is the number of resource types.

4. Features of the Simulator

Custom Scenario Input	User provides number of processes and resources, then fills in the Allocation matrix, total resources, and Request matrix.
Built-in Demo Scenarios	Three pre-loaded examples: (1) classic 2-resource deadlock, (2) no deadlock / safe state, (3) safe state with multiple resources.
System State Display	Before detection, the simulator prints the Allocation Matrix, Request Matrix, and Available vector.
Deadlock Report	After analysis, the simulator clearly states whether deadlock exists and lists the exact processes involved.
Resolution Suggestions	On detecting a deadlock, the program suggests three standard resolution strategies: process termination, resource preemption, and rollback.
Input Validation	All integer inputs are validated against accepted ranges; invalid entries prompt the user to re-enter.
Repeat Runs	After each analysis, the user can run another scenario without restarting the program.

5. Data Structures Used

Variable	Type	Purpose
allocation[i][j]	vector<vector<int>>	Units of resource j currently held by process i
request[i][j]	vector<vector<int>>	Units of resource j currently requested by process i
available[j]	vector<int>	Currently free units of resource type j
work[j]	vector<int>	Temporary work vector (copy of available, grows as processes finish)
finish[i]	vector<bool>	True if process i can complete; used to track progress in the algorithm
deadlocked	vector<int>	Indices of processes identified as deadlocked

6. How to Compile and Run

Compile (g++ on Linux/macOS/Windows with MinGW):

```
g++ -std=c++11 -o deadlock_sim deadlock_simulator.cpp
```

Run:

```
./deadlock_sim (Linux/macOS) deadlock_sim.exe (Windows)
```

No external libraries are required. The program uses only the C++ Standard Library (<iostream>, <vector>, <string>, <algorithm>).

7. Sample Output

Below is a condensed example of the simulator output for Demo 1:

```
=====
DEADLOCK DETECTION SIMULATOR
=====

===== MAIN MENU =====
1. Enter custom scenario
2. Load demo scenario
3. Exit
Choose option (1-3): 2
Demo (1-3): 1

Loaded Demo 1: Classic 2-resource, 3-process deadlock.

--- Current System State ---
Allocation Matrix: R0 R1
P0 1 0
P1 0 1
P2 0 0
```

```
Request Matrix: R0 R1
```

```
P0 0 1
```

```
P1 1 0
```

```
P2 0 0
```

```
Available Resources: R0=0 R1=0
```

```
--- Running Deadlock Detection ---
```

```
[RESULT] DEADLOCK DETECTED!
```

```
Deadlocked processes: P0 P1
```

```
Possible resolutions:
```

1. Terminate one of the deadlocked processes.
2. Preempt a resource from one process and give it to another.
3. Roll back a process to a safe checkpoint.

8. Conclusion

The Deadlock Detection Simulator demonstrates a fundamental Operating Systems concept in a clear, interactive manner. By allowing users to define their own resource allocation states or explore pre-built scenarios, the program bridges the gap between theoretical understanding and practical intuition.

The work-vector detection algorithm is both correct and efficient for small-to-medium sized systems, and the console interface keeps the implementation straightforward without sacrificing educational value. This project can be extended further by adding a Resource Allocation Graph visualizer, integration with Banker's Algorithm for deadlock avoidance, or a GUI front-end.